

calltarget(unevaluated-call-expression)

Document #: P2825R0
Date: 2023-03-15
Project: Programming Language C++
Audience: EWG
Reply-to: Gašper Ažman
<gasper.azman@gmail.com>

Contents

- [1 Introduction](#)
- [2 Motivation and Prior Art](#)
 - [2.1 Related Work](#)
- [3 Proposal](#)
 - [3.1 Interesting cases in the above example](#)
 - [3.2 Alternatives to syntax](#)
 - [3.3 Naming](#)
- [4 Possible Extensions](#)
 - [4.1 Inventing constructor free-functions](#)
 - [4.2 Inventing destructor free-forms](#)
 - [4.3 Inventing pointers to built-in functions](#)
- [5 Usecases](#)
 - [5.1 What does this give us that we don't have yet](#)
 - [5.2 That's not good enough to do all that work. What else?](#)
- [6 References](#)

§ 1 Introduction

This paper introduces a new compile-time expression into the language, for the moment with the syntax `__builtin_calltarget(postfix-expression)`.

The expression is a compile-time constant with the value of the pointer-to-function (PF) or pointer-to-member-function (PMF) that *would have been called* if the *postfix-expression* had been evaluated.

In that, it's basically a compile-time resolver.

§ 2 Motivation and Prior Art

The language already has a number of sort-of overload resolution facilities:

- `static_cast`
- assignment to a variable of a given function pointer type
- function calls (implicit) - the only one that actually works

All of these are woefully unsuitable for type-erasure that library authors (such as [P2300R6]) would actually *like* to work with. Sure, we can always indirect through a lambda:

```
template <typename R, typename Args...>
struct my_erased_wrapper {
    using fptr = R (*)(Args...);
    fptr erased = +[](my_erased_wrapper* self, auto&&... args_) -> R {
        return self->fptr(std::forward<decltype>(args_)...);
    };
};
```

This has several drawbacks:

- it introduces a whole new lambda scope
 - annoying for optimizers because extra inlining
 - annoying for debugging because of an extra stack frame
- a requirement to divine forward noexceptness by the user,
- a requirement that arguments are *convertible* (but not necessarily the same) as the ones the user specified in their infinite wisdom,
- a requirement that the return type is convertible (but not necessarily the same) as the one the user specified in their infinite wisdom,
- the forward-through-a-reference inhibits copy-elision of arguments
- inability to flatten subsetting type-erased wrappers (we can't divine an exact match in the presence of overloads) (think “subsetting vtables”)

Oh, if only we had a facility to ask the compiler what function we'd be calling and then *just have the pointer to it*.

This is what this paper is trying to provide.

§ 2.1 Related Work

§ 2.1.1 Reflection

Of course, reflection would give us this. However, reflection ([P2320R0],[P1240R1],[P2237R0],[P2087R0],[N4856]) is both nowhere close to shipping, and is far wider in scope as another decltype-ish proposal that's easily implementable today, and `std::execution` could use immediately.

Regardless of how we chose to provide this facility, it is dearly needed, and should be provided by the standard library or a built-in.

See the [Alternatives to Syntax](#) chapter for details.

§ 2.1.2 Library fundamentals TS v3

The [Library Fundamentals TS version 3](#) defines `invocation_type<F(Args...)>` and `raw_invocation_type<F(Args...)>` with the hope of getting the function pointer type of a given call expression.

However, this is not good enough to actually be able to perform that call.

Observe:

```
struct S {
    static void f(S) {} // #1
    void f(this S) {}   // #2
};
void h() {
    static_cast<void(*)>(S){S::f} // error, ambiguous
    S{}.f(S{}); // calls #1
    S{}.f(); // calls #2
    // no ambiguity for __builtin_calltarget
    __builtin_calltarget(S{}.f(S{})); // &#1
    __builtin_calltarget(S{}.f()); // &#2
}
```

A library solution can't give us this, no matter how much we try, unless we can reflect on unevaluated operands (which Reflection does).

§ 3 Proposal

We propose a new (technically) non-overloadable operator (because `sizeof` is one, and this behaves similarly):

(strawman syntax)

```
auto fptr = __builtin_calltarget(expression);
```

Where the program is ill-formed if *expression* does not call a function as its top-level AST-node (good luck to me wording this).

Examples:

```

void g(long x) { return x+1; }
void f() {} // #1
void f(int) {} // #2
struct S {
    friend auto operator+(S, S) noexcept -> S { return {}; } // #3
    auto operator-(S) -> S { return {}; } // #4
    auto operator-(S, S) -> S { return {}; } // #5
    void f() {} // #6
    void f(int) {} // #7
    S() noexcept {} // #8
    ~S() noexcept {} // #9
    auto operator->(this auto&& self) const -> S*; // #10
    auto operator[](this auto&& self, int i) -> int; // #11
    static auto f(S) -> int; // #12
    using fptr = void(*) (long);
    auto operator void(*)() const { return &g; } // #13
    auto operator<=>(S const&) = default; // #14
};
S f(int, long) { return S{}; } // #15
struct U : S {}

void h() {
    S s;
    U u;
    __builtin_calltarget(f()); // ok, &#1
    (A)
    __builtin_calltarget(f(1)); // ok, &#2
    (B)
    __builtin_calltarget(f(std::declval<int>())); // ok, &#2
    (C)
    __builtin_calltarget(f(1s)); // ok, &#2 (!)
    (D)
    __builtin_calltarget(s + s); // ok, &#3
    (E)
    __builtin_calltarget(-s); // ok, &#4
    (F)
    __builtin_calltarget(-u); // ok, &#4 (!)
    (G)
    __builtin_calltarget(s - s); // ok, &#5
    (H)
    __builtin_calltarget(s.f()); // ok, &#6
    (I)
    __builtin_calltarget(u.f()); // ok, &#6 (!)
    (J)
    __builtin_calltarget(s.f(2)); // ok, &#7
    (K)
    __builtin_calltarget(s); // error, constructor
    (L)
    __builtin_calltarget(s.S::~~S()); // error, destructor
    (M)
    __builtin_calltarget(s->f()); // ok, &#6 (not &#10)
    (N)
    __builtin_calltarget(s.S::operator->()); // ok, &#10
    (O)
    __builtin_calltarget(s[1]); // ok, &#11
    (P)
    __builtin_calltarget(S::f(S{})); // ok, &#12
    (Q)

```

```

__builtin_calltarget(s.f(S{}));           // ok,  &#12
(R)
__builtin_calltarget(s(11));              // ok,  &#13
(S)
__builtin_calltarget(f(1, 2));            // ok,  &#15
(T)
__builtin_calltarget(new (nullptr) S());  // error, not function
(U)
__builtin_calltarget(delete &s);          // error, not function
(V)
__builtin_calltarget(1 + 1);              // error, built-in
(W)
__builtin_calltarget([]{
    return __builtin_calltarget(f());
})();                                     // ok,  &2
(X)
__builtin_calltarget(S{} < S{});          // error, synthesized
(Y)
}

```

§ 3.1 Interesting cases in the above example

- resolving different members of a free-function overload set (A, B, C, D, T)
 - the (D) case is important - the short argument still resolves to the int overload!
- constructors and destructors (L, M, U, V) - see the **possible extensions** chapter.
- resolving different member of a member-function overload set (I, J, K, N, Q, R)
 - the (J) example is important - the call on u still resolves to a member function of S.
- resolving built-in non-functions (W): we could make this work in a future extension (see that chapter).
- resolving operator `>` (N and O). ^(expressions) specifies that *postfix-expressions* group left-to-right, which means the top-most postfix-expression is the call to `f()`, and not the `->`. To get to `S::operator->`, we have to ask for it explicitly.
- surrogate function call (S) - again, the top-most call-expression is the function call to `g`, so that is what is returned.
- nested calls: (X) the top-level call is a call to a function-pointer to #2, so that is what is returned.
- Synthesized operators (Y) - these are not functions that we can take pointers to, so unless we “force-manufacture” one, we can’t make this work.

§ 3.2 Alternatives to syntax

We could wait for reflection in which case we could write `call_target` roughly as

```

namespace std::meta {
    template<info r> constexpr auto call_target = []{
        if constexpr (is_nonstatic_member(r)) {

```

```

    return pointer_to_member<[:pm_type_of(r):]>(r);
  } else {
    return entity_ref<[:type_of:]>(r);
  } /* insert additional cases as we define them. */
}();
}

```

And call it as

```
auto my_expr_ptr = call_target<^f()>;
```

It's unlikely to be quite as efficient as just hooking directly into the resolver, but it does have the nice property that it doesn't take up a whole keyword.

Many thanks to Daveed Vandevoorde for helping out with this example.

§ 3.3 Naming

§ 3.3.1 Grabbing a pattern

A suggestion of an esteemed former EWG chair is that we, as a committee, grab the keyword-space prefix `__std_meta_*` and have all the functions with that prefix have unevaluated arguments.

In that case, this proposal becomes

```
__std_meta_calltarget(unevaluated-expression);
```

This is done so as to stop wringing our hands about function-like APIs that have unevaluated operands, and going for less appropriate solutions for want of a function. The naming itself signals that it's not a normal function. Its address also can't be taken, it behaves as-if `constexpr`.

It's ugly on purpose. That's by design. It's not meant to be pretty.

§ 3.3.2 Possible names

For all intents and purposes, this facility grammatically behaves in the same way as `sizeof`, except that we should require the parentheses around the operand.

We could call it something really long and unlikely to conflict, like `expression_targetof`, or `calltargetof` or `decltargetof` or `targetexpr` or `resolvetarget`.

§ 4 Possible Extensions

We could make compilers invent functions for the cases that currently aren't legal.

§ 4.1 Inventing contructor free-functions

For instance, constructor calls could “invent” a free function that is expression-equivalent to calling placement new on the return object:

```
// immovable aggregate
struct my_immovable_type {
    my_other_immovable_type x;
    some_immovable_type y = {};
};
my_immovable_type x(my_other_immovable_type{});
// note: prvalue parameters in type, since in-place construction through
// copy-elision is possible
std::same_as<my_immovable_type(*)>(my_other_immovable_type,
    some_immovable_type)>
    auto constructor_pointer =
    __builtin_calltarget(my_immovable_type(my_other_immovable_type{}));
auto y = constructor_pointer(my_other_immovable_type{});
// x and y have no difference in construction.
```

The main problem with this is that free functions have a different ABI than constructors of aggregates - they would have to expose where to construct the arguments in their signatures.

This paper is not about that, so we leave it for the future.

§ 4.2 Inventing destructor free-forms

Or, for destructors (this would make smart pointers slightly faster and easier to do):

```
std::same_as<void(*)>(S&)> auto
    dtor = [](S* x){return __builtin_calltarget(x->~S());}(nullptr);
S x;
dtor(x); // expression-equivalent to x.~S()
```

§ 4.3 Inventing pointers to built-in functions

We could invent pointers to functions that are otherwise built-in, like built-in operators:

```
__builtin_calltarget(1+1); // &::operator+(int, int)
```

§ 5 Usecases

Broadly, anywhere where we want to type-erase a call-expression. Broad uses in any type-erasure library, smart pointers, ABI-stable interfaces, compilation barriers, task-queues, runtime lifts for double-dispatch, and the list goes on and on and on and ...

§ 5.1 What does this give us that we don't have yet

Two things, mainly:

— no need for indirection through a lambda when resolving overload sets

- ... which allows us to use copy-elision to construct function arguments in type-erased interfaces, which is currently impossible without accurate type information from the user.

§ 5.2 That's not good enough to do all that work. What else?

Together with [P2826R0], the two papers constitute the ability to implement *expression-equivalent* in many important cases (not all, that's probably impossible).

[P2826R0] proposes a way for a function signature to participate in overload resolution and, if it wins, be replaced by some other function.

This facility is the key to *finding* that other function. The ability to preserve prvalue-ness is crucial to implementing quite a lot of the standard library customization points as mandated by the standard, without compiler help.

§ 6 References

[N4856] David Sankel. 2020-03-02. C++ Extensions for Reflection.

<https://wg21.link/n4856>

[P1240R1] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2019-10-08. Scalable Reflection in C++.

<https://wg21.link/p1240r1>

[P2087R0] Mihail Naydenov. 2020-01-12. Reflection Naming: fix reflexpr.

<https://wg21.link/p2087r0>

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.

<https://wg21.link/p2237r0>

[P2300R6] Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach. 2023-01-19. `std::execution``.

<https://wg21.link/p2300r6>

[P2320R0] Andrew Sutton, Wyatt Childers, Daveed Vandevoorde. 2021-02-15. The Syntax of Static Reflection.

<https://wg21.link/p2320r0>

[P2826R0] Gašper Ažman. Replacement functions.

<https://wg21.link/P2826R0>